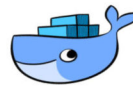


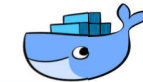
Docker Image

- Images are read-only templates that you build from a set of instructions written in your Dockerfile.
- Images define both what you want your packaged application and its dependencies to look like *and* what processes to run when it's launched.
- The Docker image is built using a Dockerfile.
- Each instruction in the Dockerfile adds a new "layer" to the image, with layers representing a portion of the images file system that either adds to or replaces the layer below it.
- Layers are key to Docker's lightweight yet powerful structure.
- Docker uses a Union File System to achieve this:



BITS Pilani, Pilani Campus

Dockerfile



A Dockerfile is where you write the instructions to build a Docker image.

These instructions can be:

- **RUN apt-get y install some-package:** to install a software package
- **EXPOSE 8000:** to expose a port
- **ENV ANT_HOME /usr/local/apache-ant** to pass an environment variable and so forth.

Once you've got your Dockerfile set up, you can use the **docker build** command to build an image from it.

BITS Pilani, Pilani Campus

Ingredients of a Dockerfile

- First line: **FROM**
- Meta-data can be added as **LABEL**
- Environment variables can come in handy with **ENV**
- You can **ADD** or **COPY** files to the image
- It is possible to **RUN** commands in the image
- Do not forget to **EXPOSE** network ports
- Containers start their life with a **CMD** or an **ENTRYPOINT**

```
FROM debian:stretch-slim
LABEL maintainer="NGINX Docker Maintainers <docker-maint@nginx>"
ENV NGINX_VERSION 1.15.2-1~stretch
ENV NJS_VERSION 1.15.2.0.2-1~stretch
RUN set -x \
    \&& apt-get update \
    \&& apt-get install --no-install-recommends --no-inst \
    # ... some lines removed for better readability...
    echo "deb https://nginx.org/packages/mainline/debian" \
    \&& apt-get update \
    # ...
    \&& apt-get install --no-install-recommends --no-ins \
    $nginxPackages \
    gettext-base \
    # ...
    # forward request and error logs to docker log collector RUN in sf
    /dev/stdout /var/log/nginx/access.log \
    \&& in sf /dev/stderr /var/log/nginx/error.log
EXPOSE 80
STOPSIGNAL SIGTERM
CMD ["nginx", "-g", "daemon off;"]
```

Each line is executed in a temporary container which gets persisted as a new layer on top of the existing layers.

- Remember the layer limit – keep it **short** and **simple**.

BITS Pilani, Pilani Campus

Dockerfile directives (1)

FROM

- first line in a Dockerfile
- specifies which image to build upon / to extend
- special case: **FROM scratch** starts from scratch without any previous image

Examples:

- **FROM debian:jessie**
– will pull the a Debian image from Docker Hub and use it as base image
- **FROM scratch**
– will start with an empty image

ENV

- Sets an environment variable
- Will be present during the build process and in the final image

Examples:

- **ENV http_proxy http://proxy:8080**
– will set the http_proxy environment while building the image and in the finished image

BITS Pilani, Pilani Campus

Dockerfile directives (2)

LABEL

- sets meta-data of an image
- labels are key-value pairs

Examples:

- **LABEL maintainer="Homer Simpson"**
– adds a piece of meta-data to the image specifying the maintainer

ADD

- adds a file from the build-context to the image
- can download files from a URL
- will automatically extract archives

Examples:

- **ADD rootfs.tar.bz2 /**
– will extract the contents of rootfs.tar.bz2 to the root directory / of the image
- **ADD http://plx172:1080/pic.jpg /usr/share/pics**
– will download a picture from plx172 and place it into the image at /usr/share/pics

BITS Pilani, Pilani Campus

Dockerfile directives (3)

COPY

- copies a file from the build-context to the image
- no additional magic like the ADD command
- does not copy files inside the image

Examples:

- **COPY default.conf /etc/nginx/conf.d/**
– will copy the file default.conf into the image at /etc/nginx/conf.d/default.conf

RUN

- runs the specified command in the image
- command binary and shell must be present in the image

Examples:

- **RUN apt-get update**
– will run the Debian package manager and build its package cache inside the image
- **RUN cp /etc/nginx/nginx.conf /root**
– will copy the file /etc/nginx/nginx.conf within the image to the directory /root within the image

BITS Pilani, Pilani Campus

Dockerfile directives (4)

EXPOSE

- exposes a port
- exposed ports can be automatically mapped by Docker

Examples:

- **EXPOSE 80**
– will expose port 80 so that “docker run -P” can automatically map it to a port on the host

CMD

- specifies the command to run when a container is created from the image
- all but the last CMD directive are ignored
- can be overridden upon container instantiation

Examples:

- **CMD /bin/helloworld**
– will run a hello-world program when the container is started. This form is executed in a shell
- **CMD ["nginx", "-c", "daemon off;"]**
– will run the nginx binary with the arguments “-c” and “daemon off;” This form is directly executed via a system call (without shell)

BITS Pilani, Pilani Campus

Dockerfile directives (5)

ENTRYPOINT

- sets the program to be directly run when a container is created
- can be overridden with the option “--entrypoint”.

Examples:

- **ENTRYPOINT**
- **["/usr/bin/nginx"]**
– starts nginx directly without using a shell to start it. (Note the brackets)

USER

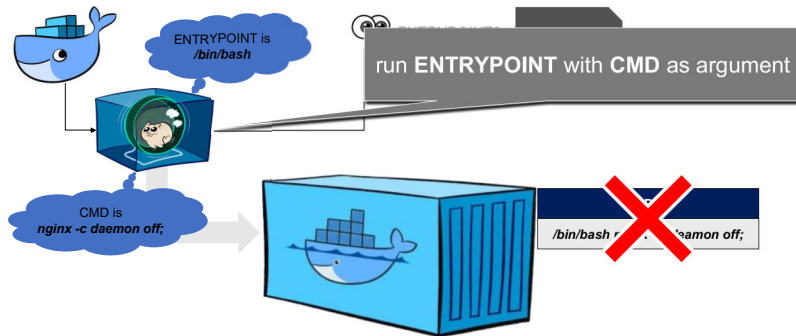
- Set the active user & group for following commands
- Specified user is valid for RUN as well as CMD & Entrypoint
- Prerequisite: the target user / group has to exist

Examples:

- **RUN adduser -D -s /sbin/nologin nginx**
– will create a new user with name “nginx”
- **USER nginx**
– will switch from current user to “nginx”. Valid for all following commands such as RUN but also when starting a new container later

BITS Pilani, Pilani Campus

Entrypoints, CMDs and the container lifecycle



BITS Pilani, Pilani Campus

Running a container as root

user "root" within the container is the same as outside

```
# whoami
>> root
# id root
>> uid=0(root) gid=0(root)
# touch /hostfs/test
```

Via volumes files owned by "root" can be created or modified on the host

```
$: ps aux
...
root /usr/bin/containerd
root \_ containerd-shim -namespace moby
root \_ /bin/sh
root \_ whoami
```

BITS Pilani, Pilani Campus

USER – non-root is best practice!

Use a different user within the container to limit access to the host

```
# whoami
>> app-user
# id app-user
>> uid=1001(app-user) gid=1001(app)
# touch /hostfs/test
Permission denied.
```

Regular permission checks apply

```
$: ps aux
...
root /usr/bin/containerd
root \_ containerd-shim -namespace moby
1001 \_ /bin/sh
1001 \_ whoami
```

BITS Pilani, Pilani Campus

User Namespace: remap

user "root" within the container is mapped to a different UID

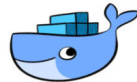
```
# whoami
>> root
# id root
>> uid=0(root) gid=0(root)
# touch /hostfs/test
```

Docker can be configured to automatically remap user & groups

```
$: ps aux
...
root /usr/bin/containerd
root \_ containerd-shim -namespace moby
231072 \_ /bin/sh
231072 \_ whoami
```

BITS Pilani, Pilani Campus

Docker multistage builds



```
1 ## Dockerfile with multistage build
2
3 # stage 1
4 FROM golang:1.7.3 as builder
5
6 WORKDIR /go/src
7
8 RUN go get -d -v golang.org/x/net/html
9 COPY app.go .
10 RUN go build -a -installsuffix cgo -o app .
11
12 # stage 2
13 FROM alpine:latest
14
15 RUN apk --no-cache add ca-certificates
16 COPY --from=builder /go/src/app .
17
18 CMD [ "./app" ]
19
```

First build stage with dedicated stage name ("as **builder**")

Second stage is done in a container totally separate from first stage

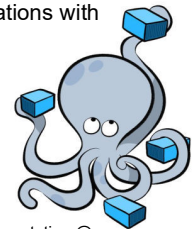
Files get copied from the container of the first build stage

BITS Pilani, Pilani Campus

Docker Compose



- Compose is a tool for defining and running multi-container Docker applications.
- What is docker compose?
- Docker Compose is a tool for defining and running complex applications with Docker.
- Define a multi-container application in a single file
- Spin your application up in a single command



Complete documentation @
<https://docs.docker.com/compose/overview>

BITS Pilani, Pilani Campus

Features



- Multiple isolated environments on a single host
- Preserve volume data when containers are created
- Only recreate containers that have changed
- Variables and moving a composition between environments
- Multiple compose files



BITS Pilani, Pilani Campus

Installation



- ★ Windows / Mac - Docker Compose is included ☺
- ★ Linux:

```
sudo curl -L
https://github.com/docker/compose/releases/download/2.27.1/docker-
compose-$(uname -s)-$(uname -m) -o /usr/local/bin/docker-compose
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

Test the installation

```
docker-compose --version
```

BITS Pilani, Pilani Campus



Commands

- ★ Create and start all the containers listed in the “docker-compose.yml”

```
$ docker-compose up -d
```

- ★ List all the containers belong to the compose environment instance:

```
$ docker-compose ps
```

- ★ Sets the number of containers:

```
$ docker-compose scale web=3
```

BITS Pilani, Pilani Campus

Commands

build	Build or rebuild services
help	Get help on a command
kill	Kill containers
logs	View output from containers
port	Print the public port for a port binding
ps	List containers
pull	Pulls service images
rm	Remove stopped containers
run	Run a one-off command
scale	Set number of containers for a service
start	Start services
stop	Stop services
restart	Restart services
up	Create and start containers

BITS Pilani, Pilani Campus

Before Docker Compose

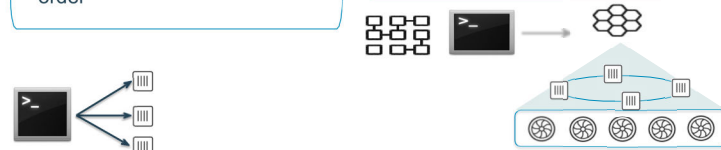
```
$ docker run -d -it --name redis redis
$ docker run -d -it --name postgres linhmtran168/postgres
$ docker run -d -it --name web \ -v ~/Dev/gitlab.com/linhmtran168/test-
project:/var/www/html \ --link postgres:db --link redis:redis linhmtran168/php-web
$ docker run -d -it -p 80:80 --name nginx \ --link web:web --volumes-from web
linhmtran168/php-nginx
$ docker run -d -it --name node --link web:web \ --volumes-from web
linhmtran168/gulp-bower
```

BITS Pilani, Pilani Campus

Docker Compose: Multi Container Applications

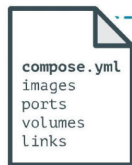
- Build and run one container at a time
- Manually connect containers together
- Must be careful with dependencies and start up order

- Define multi container app in compose.yml file
- Single command to deploy entire app Handles container dependencies
- Works with Docker Swarm, Networking, Volumes, Universal Control Plane



BITS Pilani, Pilani Campus

Docker Compose: Multi Container Applications



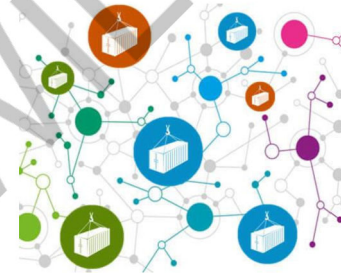
```
version: '2' # specify docker-compose version
# Define the services/containers to be run
services:
  angular: # name of the first service
    build: client # specify the directory of the Dockerfile
    ports:
      - "4200:4200" # specify port forwarding

  express: #name of the second service
    build: api # specify the directory of the Dockerfile
    ports:
      - "3977:3977" #specify ports forwarding

  database: # name of the third service
    image: mongo # specify image to build container from
    ports:
      - "27017:27017" # specify port forwarding
```

BITS Pilani, Pilani Campus

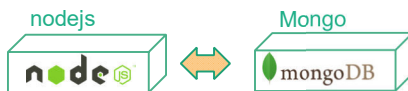
Docker Compose: Scale Container Applications



BITS Pilani, Pilani Campus

Docker Compose use case example

Lets chat app



docker-compose.yml

```
Web:
  build: .
  ports:
    - "9000:9000"
  links:
    - mongo

mongo:
  image: mongo:latest
```

BITS Pilani, Pilani Campus

Docker Compose V 2

docker-compose.yml

```
version: '2'

Services
Networks
Volumes
```

docker-compose.yml

```
version: '2'

Services:
web:
  build:
    context: .
    dockerfile: Dockerfile
  image: demochatweb:0.1
mongo:
  image: mongo
```

BITS Pilani, Pilani Campus



Networks

docker-compose.yml

```
version: '2'
services:
  service1:
    image: ubuntu
    command: sleep 3600
    networks:
      - internal1
      - default
  service2:
    image: ubuntu
    command: sleep 3600
    networks:
      - internal1
  service3:
    image: ubuntu
    command: sleep 3600
    networks:
      - default
networks:
  internal1:
    driver: bridge
```

Volumes

docker-compose.yml

```
version: '2'
services:
  service1:
    image: ubuntu
    command: sleep 3600
    volumes:
      - data:/data
  service2:
    image: ubuntu
    command: sleep 3600
    volumes:
      - data:/data
volumes:
  data:
    driver: local
```

BITS Pilani, Pilani Campus

Docker-Compose - Benefits

- 1. Easy setup and reproducibility:** With a single YAML file, you can define the entire environment for your application, including all the necessary services and their configurations. This makes it easy to share and reproduce the application setup across different environments.
- 2. Efficient development workflow:** Docker Compose allows developers to define the entire application stack locally and run it with a single command. This enables them to test, debug, and iterate on their code without worrying about the complexities of setting up and configuring multiple containers manually.
- 3. Scalability and orchestration:** Docker Compose can be integrated with other orchestration tools such as Docker Swarm or Kubernetes to scale and manage your application across multiple hosts or in a cluster environment.

BITS Pilani, Pilani Campus

Swarm Basics

Run multiple commands utilizing supervisor (or similar)

Networking (map host ports to the container, etc)

Data Volumes / Data Volume Containers

- Persist data
- Share data among containers

Docker swarm (native clustering)



BITS Pilani, Pilani Campus

Container Orchestration

- Containers are ephemeral by nature.
- They stop when process inside them finishes, or ends because of an error.
- Also a single container per service may not be sufficient enough to handle the growing traffic to the application.
- In all the above scenarios we need a tool that can bring up the stopped containers or spin up new one's to handle the growing traffic and to ensure Load balancing & High availability of the application all the time.
- This is where container orchestration comes into play!!!

What is container orchestration?

Container orchestration is all about managing the lifecycles of containers, especially in large, dynamic environments.

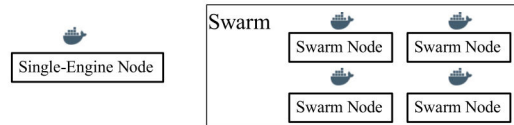
- ✓ Provisioning and deployment of containers
- ✓ Scaling up or removing containers to spread application load evenly across host infrastructure
- ✓ Movement of containers from one host to another if there is a shortage of resources in a host, or if a host dies
- ✓ Load balancing of service discovery between containers

BITS Pilani, Pilani Campus



Docker Swarm

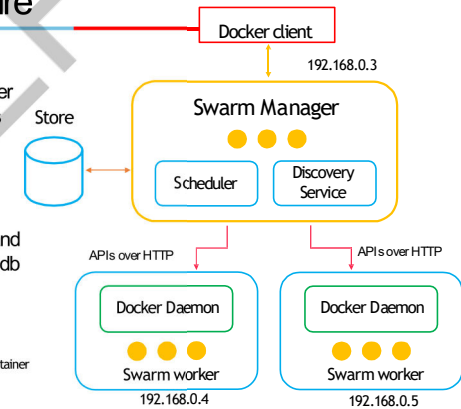
- Orchestrating thousands of containers
- Swarm: A group of nodes collaborating over a network
- Two modes for Docker hosts:
 - Single Engine Mode: Not participating in a swarm
 - Swarm Mode: Participating in a Swarm
- A service may run on a swarm
- Each swarm has a few managers that dispatch tasks to workers. Managers are also workers (i.e., execute tasks)



BITS Pilani, Pilani Campus

Docker Swarm Architecture

- Manager: manages the cluster
- Worker: runs tasks assigned by scheduler
- Scheduler: schedules containers onto nodes depending upon rules and filters
- Discovery service: helps Swarm Manager discover new nodes and fetch the available nodes
- Store: stores state of cluster, like cluster and swarm services info. Essentially a key-value db



BITS Pilani, Pilani Campus

Docker Swarm Architecture

- The managers select a leader, who really keeps track of the swarm
- Assigns tasks, re-assigns failed worker's tasks, ...
- Other managers just monitor passively and re-elect a leader if leader fails
- Services can be scaled up or down as needed
- Several Docker commands:
 - **docker service**: Manage services
 - **docker swarm**: Manage swarms
 - **docker node**: Manage nodes

BITS Pilani, Pilani Campus

IMP Note to Self





BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Thank you

BITS Pilani, Pilani Campus





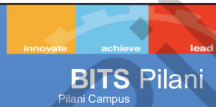
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



BITS Pilani
Pilani Campus

Introduction to DevOps

Slides edited /compiled by Dr. Udhayakumar
for DevOps faculty team.



CSIW ZG515, Introduction to DevOps CS 14 : Containerization

IMP Note to Self



IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT.**

BITS Pilani, Pilani Campus



Session 14 – Hand out Syllabus

Module 9 Virtualization and Containerization

Pre-CH/CS	RL 9.2	RL9.2 Micro-services and Function as a Service	
		- RL9.2.1 Overview of Micro-services - RL9.2.2 Introduction to AWS Lambda	
During CH/CS	CS14	- Micro-services and Containerization - Function-As-A-Service (AWS Lambda)	T1 - Chapter 9, R1 - Chapter 23

BITS Pilani, Pilani Campus

Agenda

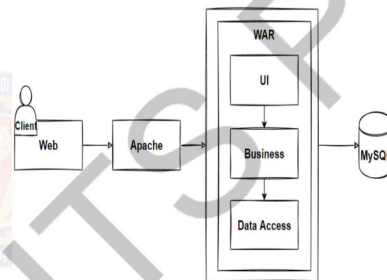
Overview of Microservices

- Introduction to Microservices
- List of Organizations who are using Microservices
- Common characteristics for Microservices
- Need of Microservices for DevOps
- DevOps and Microservices
- Benefits of DevOps and Microservices

Monolithic Architecture

Introduction

- A Monolith is a software application that typically implements all the modules and features in a single code base.
- Each features say dashboard/account summary/payments can be separate modules, but all these modules are packaged as single achieve file and deployed .
- Monolith application is implement as N-Tier application consisting on Presentation/Application/Database layer.
- This architecture reigned supreme in web development, with a whopping 94 percent of developers favoring it for its straight forwardness and speed .



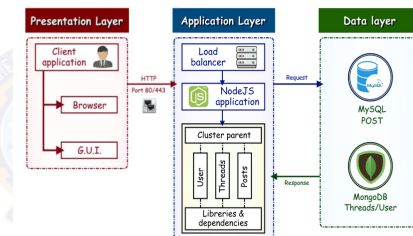
Monolithic Architecture

Advantages/Disadvantage

- **Simplicity** : All in one
- **Easy to Deployment** : Single war/jar were being deployed into application server like tomcat/jboss/.net servers.
- **Easy to debug/monitor**
- **Easy to Maintain/Manage code base**

Disadvantage

- Code base can become so big to Manage.
- Difficult to work in big teams
- Not Flexible to scale/resilient
- High Coupling and restricted to single technologies.



Monolithic architecture

Complexity and Challenges

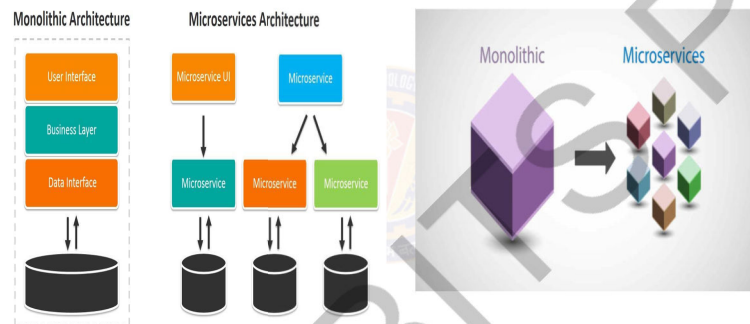
- **Scalability** : is paramount in today's tech-driven landscape. The monolithic design, resembling a single, massive container, makes scaling the entire application a daunting task.
- **Availability** : Setup, a server failure can lead to a complete application crash, particularly during peak periods, rendering the system unresponsive.
- **Performance**: Monolithic architecture struggles with performance as applications grow larger. A single server error can compromise the entire system's performance.
- **Infrastructure**: Monolithic architecture often falls short in terms of infrastructure reliability, struggling with aspects like fault tolerance, failure recovery, and service discovery.
- **Business Agility**: Delivering products through a monolithic architecture can be time-consuming and inflexible, failing to keep pace with market demands. For instance, implementing a custom payment plan within a monolithic codebase is cumbersome due to its large, unified structure.

Why Migration to Microservice architecture

- Monolithic Application can outgrow its monolithic architecture and become an obstacle to rapid, frequent and reliable software delivery.
- When the application becomes large and complex and is developed by many teams, becomes difficult to scale, when business increases.
- Entire application will not be agile/resilient for structural changes.
- Conflict occurs when big teams is working on the same code base.
- Frequent releases will be difficult ,as conflicts will occurs when merging from multiple branches to main branch.
- For example, its deployment pipeline become a bottleneck. When this occurs, you should consider **migrating to microservices**.
- Migrating to Microservices will be structure changes on Architecture/ Design, migration should be done functionality by functionality.

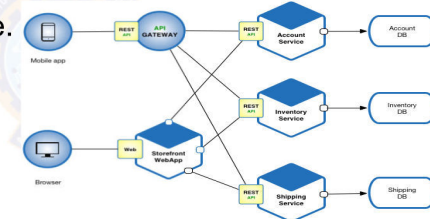
<https://microservices.io/>

Monolithic → Microservices Architecture



What are microservices?

- Microservices (or microservices architecture) is an architectural style that structures an application as a collection of services that are:
 - Independently deployable.
 - Loosely coupled.



Microservices

Principles

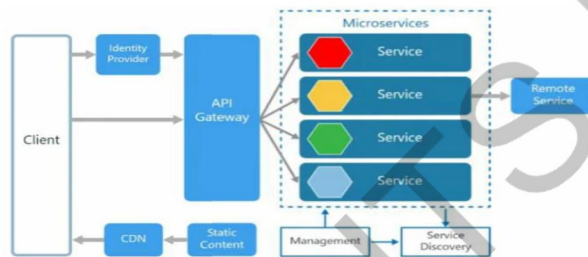
- Services are typically organized around business capabilities. Where services are separated by boundaries referred as Bounded Context.
- Service can have their own technology stack, which includes database and data management model;
- communicate between one another using REST client APIs, event streaming, and message brokers.
- Each Services can be own by small team, and their scope lies within that services only.
- Each Service can be scaled independently, reducing the cost associated with having to scale entire applications because a single feature might be facing too much load.

Features of Microservices

- API Gateway** : Is the single entry point to communicate with Microservices. All request from different client are routed to specific Microservices, as per functionality invoked.
- Discovery Service** : It registers all Microservices, enabling MS synchronous communication using Rest Client. But they can also communication Asynchronously with Brokers like kafka/JMS.
- Config Server** : All Microservices can externalize their environment properties in this server, enabling quick dynamic update to the configured properties.
- Fault Tolerant** : enabling Microservices to serve the request, even when Microservices is down/ issues wrt N/W.
- Stateless Nature** : Every Microservices is stateless, which enable them to replicate and scale faster.
- CI/CD** : Since Microservices are small services, integration and deployment is quick.

Microservices

Architecture



Bounded Context

Microservices are independent of the underlying architecture of other microservices

Loosely Coupled

Any change to one service should not affect the others.

Feature Focused

The scope is 'micro' which means it solves only 1 function

Independent

They are not dependant on the underlying architecture.

Multiple Code Base

Each microservice has its own code base

Microservices Architecture Benefits



Companies Using Microservices



DevOps Tools And Technologies To Manage Microservices

Monolith	Microservices	Docker	Kubernetes	OpenShift	Istio
Benefits Simple to Develop. Simple to Deploy. Simple Testing.	Benefits Better Scaling. New technologies. Frequent Releases.	Benefits Faster Deployment. Faster start time. Lightweight Image.	Benefits Storage options. Auto-scaling. Self-healing.	Benefits OOTB Webconsole. OOTB Build Images. Built in Jenkins. Build Images and store in Registry.	Benefits Tracing. Circuit Breaker. Canary Release. Dark Launches. Telemetry.
Challenges Hard to scale. Too Large/Complex. Redeploy the entire application on each update.	Challenges Slower Deployment. Service Discovery. Complex Configs. Load Balancing. Central Logging.	Challenges No Storage Option. No Autoscaling. No health-checks. Networking is hard.	Challenges Hard to install. No Building Image. No built in Jenkins. Separate install for Dashboard.	Challenges Requires additional API Management. Limited installation options.	Challenges Added Complexity. Adds Overhead. Required Expertise.

A brief introduction to the tools used in the above figure:

- **Docker** is a Open Source Containerization Software tool designed to make it easier to create, deploy, and run applications by using containers. Containers allow a developer to package an application with all its dependencies, such as libraries and environment and deploys it as one package.
- **Kubernetes** is an open-source container orchestration software for automating application deployment, scaling, and management.
- **OpenShift PaaS platform** is a combination of Kubernetes and Docker clusters. The architecture of OpenShift is designed in such a way that it can support and manage Docker containers, which are hosted on top of all the layers using Kubernetes.
- **Istio** reduces the complexity of managing microservices deployments by providing a uniform way to secure, connect, and monitor microservices.

Reference

- <https://www.hcltech.com/blogs/devops-tools-and-technologies-manage-microservices>
- <https://aws.amazon.com/microservices/>
- <https://microservices.io/>
- <https://dzone.com/articles/what-is-microservices-an-introduction-to-microserv>
- <https://www.ibm.com/topics/microservices>
- <https://www.mendix.com/blog/the-microservices-you-need-for-devops/>
- <https://www.mulesoft.com/resources/api/what-are-microservices>



What Is Serverless Computing?

Serverless computing is an application deployment paradigm that allows applications to run on-demand, consuming only the resources required to execute them.

Serverless



- Serverless computing is an application development and execution model that enables developers to build and run application code without provisioning or managing servers or back-end infrastructure.
- Provisioning and Managing of Server is done by the cloud provider, enabling application infrastructure to scale as per the needed demand.
- All infrastructure management task like patch updates, security management, capacity management and monitoring, client provider is solely responsible.
- Serverless lets developers to focus on implementing the best front-end application code and business logic, where as Cloud provides take care of infrastructure around the application's ie deploying to container

Never pay for idle capacity



- With serverless, developers never pay for idle capacity.
- The cloud provider spins up and provisions the required computing resources on demand when the code executes, and spins them back down again—called 'scaling to zero'—when execution stops.
- The billing starts when execution starts, and ends when execution stops; typically, pricing is based on execution time and resources required.



Serverless does not mean 'no servers'

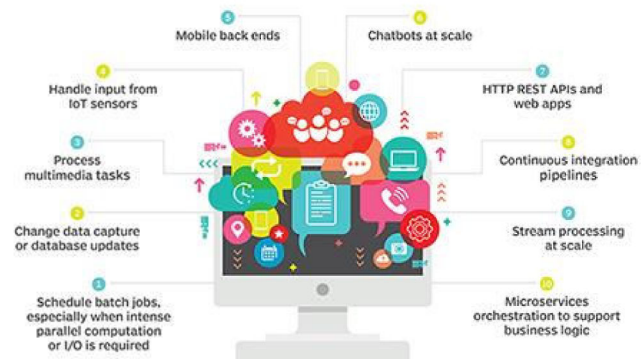


- The name notwithstanding, there are most definitely servers in serverless computing.
- 'Serverless' describes the developer's experience with those servers—they are invisible to the developer, who doesn't see them, manage them, or interact with them in any way.
- Today every leading cloud service provider offers a serverless platform including Amazon Web Services (AWS Lambda), Microsoft Azure (Azure Functions), Google Cloud (Google Cloud Functions) and IBM Cloud (IBM Cloud Code Engine).
- Together serverless computing, [microservices](#) and [containers](#) form a triumvirate of technologies typically considered to be at the core of [cloud-native](#) application development.





10 common uses for serverless platforms



AWS Lambda

AWS Lambda

What is AWS Lambda?

- AWS Lambda is a compute service that lets you run code without provisioning or managing servers
- AWS Lambda executes your code only when needed and scales automatically
- You pay only for the compute time you consume - there is no charge when your code is not running
- Often referred to as "serverless" computing. AWS Lambda allows developers to think only about the code, and not have to worry about the layers below
- You can think of PaaS service to co-relate

AWS Lambda

Function as a Service [FaaS]

- Microservices and AWS Lambda
- Microservices where applications are broken into much smaller pieces
- To enable better scalability, more agile development
- Similarly AWS Lambda allows developers to deploy a single function at a time
- Once Deployed, Lambda will manage the operational aspects of that function, from scaling to high availability
- AWS Lambda is referred as Function As A Service



AWS Lambda Concepts

Function:

A function is a program or a script which runs in AWS Lambda. Lambda passes invocation events into your function, which processes an event and returns its response.

Runtimes:

Runtime allows functions in various languages which runs on the same base execution environment. This helps you to configure your function in runtime. It also matches your selected [programming language](#).

Event source:

An event source is an AWS service, such as Amazon SNS, or a custom service. This triggers function helps you to executes its logic.

Lambda Layers:

Lambda layers are an important distribution mechanism for libraries, custom runtimes, and other important function dependencies. This AWS component also helps you to manage your development function code separately from the unchanging code and resources that it uses.

Log streams:

Log stream allows you to annotate your function code with custom logging statements which helps you to analyse the execution flow and performance of your AWS Lambda functions.

How does AWS Lambda work?

- Each Lambda function runs in its own container.
- When a function is created, Lambda packages it into a new container and then executes that container on a multi-tenant cluster of machines managed by AWS.
- Before the functions start running, each function's container is allocated its necessary RAM and CPU capacity.
- Once the functions finish running, the RAM allocated at the beginning is multiplied by the amount of time the function spent running.
- The customers then get charged based on the allocated memory and the amount of run time the function took to complete.

Why is AWS Lambda an essential part of the Serverless architecture?

- When building Serverless applications, AWS Lambda is one of the main candidates for running the application code. Typically, to complete a Serverless stack you'll need:
 - a computing service;
 - a database service; and
 - an HTTP gateway service.
- Lambda fills the primary role of the compute service on AWS.
- It also integrates with many other AWS services and, together with API Gateway, DynamoDB and RDS, forms the basis for Serverless solutions for those using AWS.
- Lambda supports many of the most popular languages and runtimes, so it's a good fit for a wide range of Serverless developers.

Steps of AWS lambda

- Step 1:** First upload your AWS Lambda code in any language supported by AWS Lambda. [Java](#), Python, Go, and C# are some of the languages that are supported by AWS Lambda function.
- Step 2:** These are some AWS services which allow you to trigger AWS Lambda.
- Step 3:** AWS Lambda helps you to upload code and the event details on which it should be triggered.
- Step 4:** Executes AWS Lambda Code when it is triggered by AWS services:

Steps of AWS lambda.....

Step 5: AWS charges only when the AWS lambda code executes, and not otherwise.

This will happen in the following scenarios:

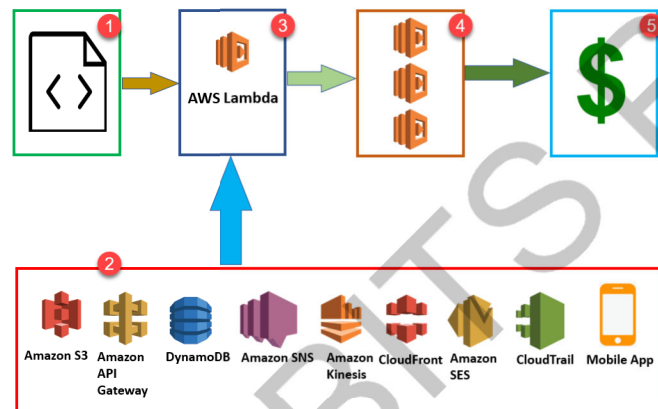
- Upload files in an S3 bucket
- When HTTP get/post endpoint URL is hit
- For adding/modifying and deleting Dynamo DB tables
- In the process of data streams collection
- Push notification
- Hosting of website
- Email sending

Note: You should remember that you will charge for AWS services only when the AWS Lambda code executes, else you don't need to pay anything.

Events that Trigger AWS Lambda

Here, are Events which will be triggered when you use AWS Lambda.

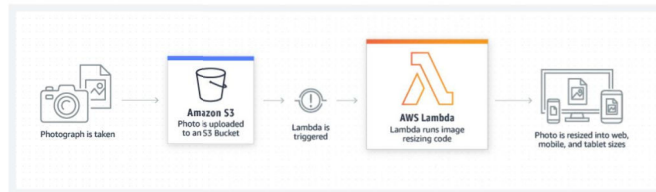
- Insert, updating and deleting data Dynamo DB table
- To include push notifications in SNS
- To search for log history in CloudTrail
- Entry into an S3 object
- DynamoDB can trigger AWS Lambda whenever there is data added, modified, and deleted in the table.
- Helps you to schedule the event to carry out the task at regular time pattern.
- Modifications to objects in S3 buckets
- Notifications sent from Amazon SNS.
- AWS Lambda can be used to process the CloudTrail logs
- API Gateway allows you to trigger AWS Lambda on GET/POST methods.



AWS Lambda Scenerio's

- File Processing : S3 to trigger Lambda
- Steam processing: Kinesis triggers Lambda
- Web Application
- IoT backends
- Mobile backends

File Processing : S3 to trigger Lambda



Use Amazon Simple Storage Service (Amazon S3) to trigger AWS Lambda data processing in real time after an upload, or connect to an existing Amazon EFS file system to enable massively parallel shared access for large-scale file processing.

Stream processing: Kinesis triggers Lambda

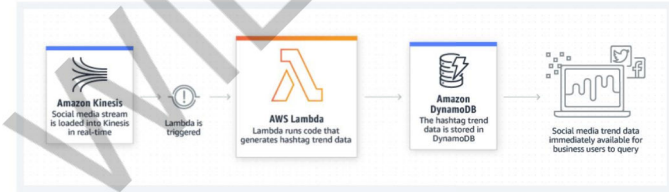


Diagram showing how Serverless stream processing works. Social media stream is loaded into Amazon Kinesis, then Lambda is triggered. Lambda runs code that generates hashtag trend data, and the data is stored in DynamoDB for easy querying.

Web Application

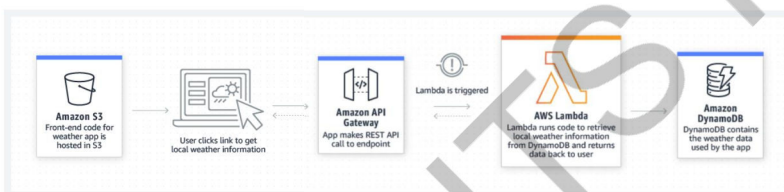
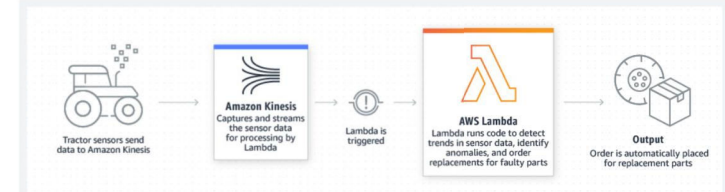


Diagram showing how Amazon S3, API Gateway, AWS Lambda, and DynamoDB work together to retrieve weather data for a web or mobile application.

IoT backends



Build serverless backends using AWS Lambda to handle web, mobile, Internet of Things (IoT), and third-party API requests.



Mobile backends



Diagram showing how Amazon API Gateway, AWS Lambda, and Amazon SNS work together to help users receive status updates notifications in a mobile application.

Thank you



BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



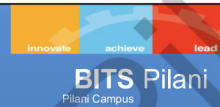
Introduction to DevOps

Slides edited /compiled by Dr. Udhayakumar
for DevOps faculty team.

IMP Note to Self



2



CSIW ZG515, Introduction to DevOps CS 15 : Kubernetes, DevOps on Cloud & AWS Lambda

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT.**

4

BITS Pilani, Pilani Campus



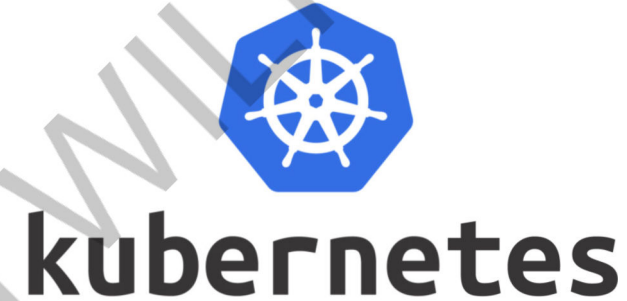
Session 15 – Hand out Syllabus

Module 9 Virtualization and Containerization

Pre-CH/CS			
During CH/CS	CS15	- Current Trends- Kubernetes, - DevOps on Cloud, - Function-As-A-Service (AWS Lambda)	T1 - Chapter 9, R1 - Chapter 23
Post-CH/CS	HW/Lab		
Lab Reference			

5

BITS Pilani, Pilani Campus



6

What is Kubernetes?

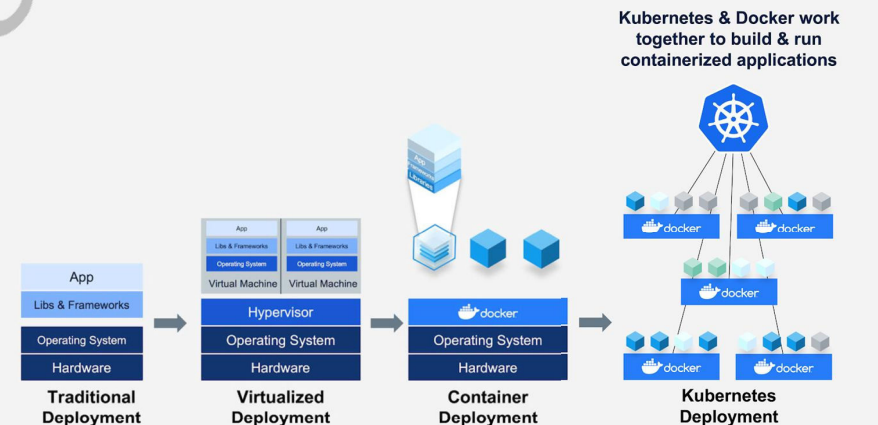
Kubernetes or K8 is an open-source container management platform [developed by Google](#) which unifies a cluster of machines into a single pool of computing resources.

With Kubernetes we can organize applications in container groups which run its docker engine to maintain applications running as per request.

Kubernetes keeps track of container applications which are deployed in cloud.

K8s is NOT a container engine like Docker.
Rather it is a container orchestrator.

7



8

Features of Kubernetes

- Schedule/manage the container as per the resource needs.
- It is Self-healing in nature, so if any container crashes, a new container is instantaneously created, replacing the crashed once.
- Horizontal scaling is achieved by observing CPU and memory metrics, scaling instances as required.
- Volume management : manages the persistent storage used by applications.
- During upgrade, new instances health is monitored and if failure occurs, It get rollbacks to previous version automatically.
- Manages application specific configuration and secrets.

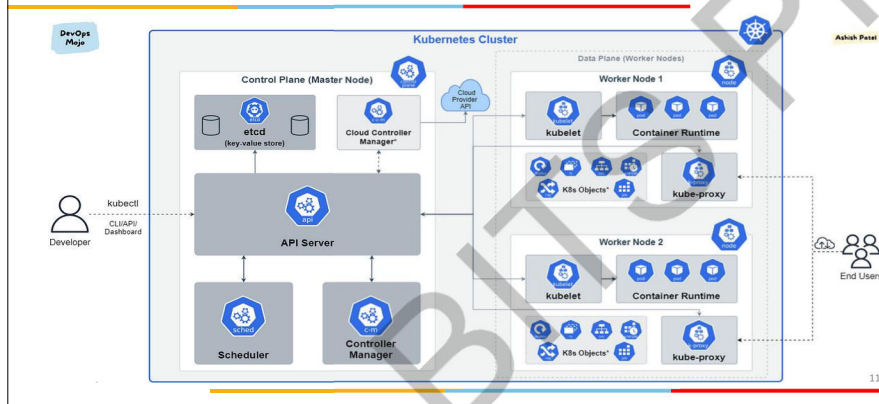
9

Use cases

- Large scale application deployments
- Managing microservices application
- CI/CD software development
- Server less computing
- [Hybrid and multi-cloud](#) deployments
- Big data processing
- Computation
- Machine learning

10

Kubernetes Architecture & Working



11

The Control Plane

- The control plane comprises several components such as
 - **Kubectl** : It is a command line tool used to interact with the Kubernetes cluster.
 - **Kube API server**: is REST server which exposes Kubernetes APIs to the external world
 - **etcd Store** : a consistent, reliable and distributed key value store (metadata store for maintaining cluster state)
 - **Kube-scheduler : Scheduler**

Kube-scheduler is responsible for scheduling Pods to worker nodes.

12

The Control Plane ...

Controller Manager

The Kube controller manager is a single process which has multiple controllers to achieve simplicity. These controllers keep a watch on changes and respond accordingly

- Node controllers are responsible for monitoring and take action when a node goes down
- Replication controller ensure that there are exact number of pods as defined for each replica set or objects for replication controller
- Endpoints controller assign for each service an endpoint object which lists the service pods
- Service account and token controllers initialize new namespaces with default service account and corresponding API access tokens

13

The Data Plane

The data plane comprises of Cluster nodes, which run containerized workloads or pods. The data plane and control plane share physical and logical machines.

Kubelet :

Kubelet is Kubernetes agent and responsible for talking to API servers and running and managing pods on the node. Kubelet is responsible for :

- Download pod secrets from API server
- Volume mounts
- Pod container running via container runtime interface (CRI)
- Report status of each Pod and node
- Probe container liveness

14

The Data Plane

Kube Proxy

Networking aspects are handled by this component for the node. It operates as local front for services and forwards TCP and UDP packets and discovers IP addresses of services via DNS or environment variables

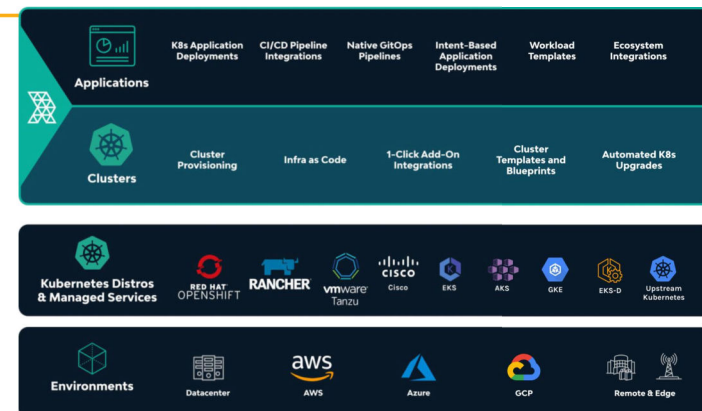
Container Runtime

Kubernetes runs containers and supports a variety of container runtimes. Docker is supported initially. Kubernetes runs containers via an interface known as CRI which is based on gRPC.



15

Kubernetes Distro

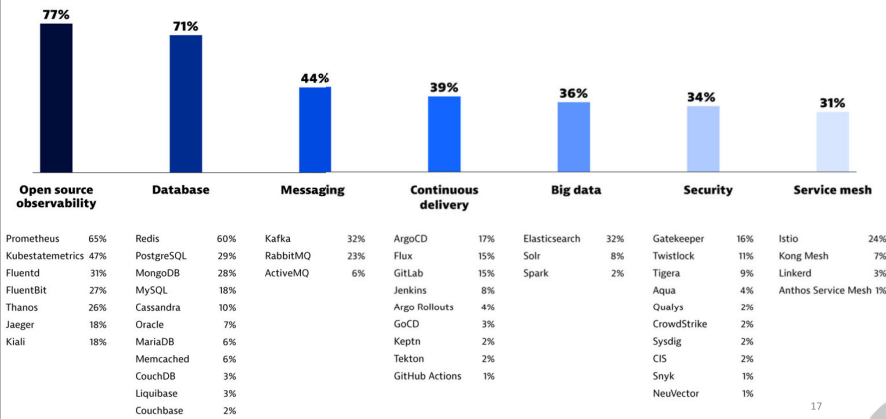


16



Technologies used in Kubernetes environments

Percentage of organizations



17

What is OpenShift?



[Red Hat OpenShift](#) is a cloud-based Kubernetes platform that helps developers build applications.

It offers automated installation, upgrades, and life cycle management throughout the container stack — the operating system, [Kubernetes](#) and cluster services, and applications — on any cloud.

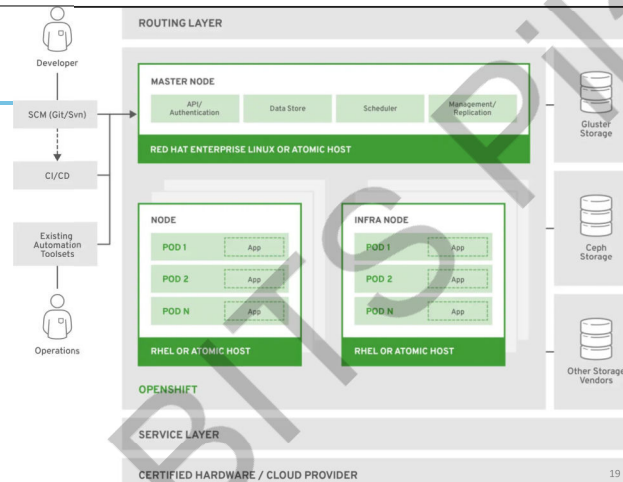
OpenShift gives organizations the ability to build, deploy, and scale applications faster both on-premises and in the cloud.

It also protects your development infrastructure at scale with enterprise-grade security.

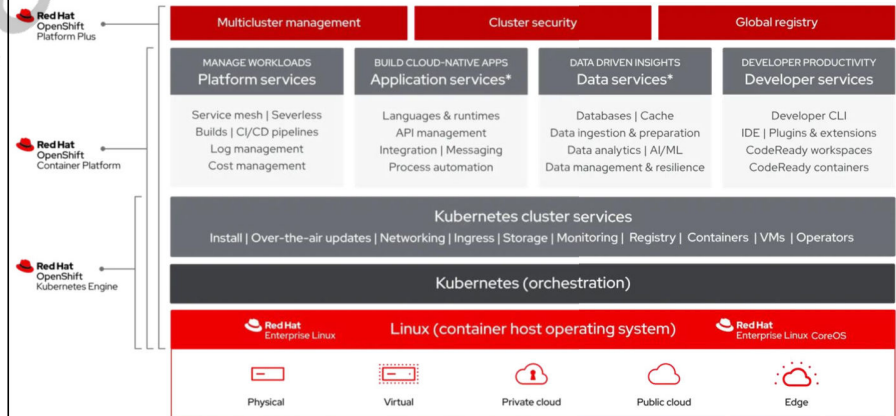
18

OpenShift

OpenShift 3.0 is built on the most popular Linux container technology, **Docker**.



19



Red Hat OpenShift includes supported runtimes for popular languages/frameworks/databases. Additional capabilities listed are from the Red Hat Application and Data Services portfolio.

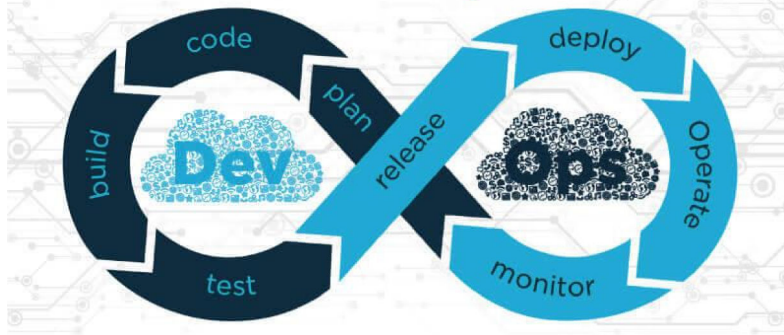
20





DevOps and Cloud

The Match that Drives Today's Businesses



21

Cloud DevOps



DevOps processes can be very agile when implemented correctly, but they can easily grind to a halt when facing the limitations of an on-premise environment.

For example, if an organization needs to procure and install new hardware in order to start a new software project or scale up a production application, it causes needless delays and complexity for DevOps teams.

Cloud infrastructure offers an important boost for DevOps and facilitates scalability.

The cloud minimizes latency and enables centralized management via a unified platform for deploying, testing, integrating, and releasing applications.

A cloud platform allows DevOps teams to adapt to changing requirements and collaborate across distributed enterprise environments.

Most cloud service providers offer CI/CD tools to automate DevOps processes.

22

Cloud DevOps....



Cloud DevOps solutions are often more cost-effective than on-prem automation solutions.

They facilitate governance by unifying the environments and reducing the security burden on teams.

The cloud offers greater control and application-specific tools to help DevOps teams maintain components from different locations.

DevOps teams can provision and manage cloud resources using code or via an automated infrastructure provisioning service, ensuring their projects maintain the desired momentum.

The high level of automation provided by cloud-based DevOps services helps minimize human error and streamlines repeatable tasks. Cloud services and tools let developers automate specific tasks to use their time more efficiently.

23

DevOps as a Service



DevOps as a Service is a set of cloud-based tools that enable collaboration between an organization's development and operations teams.

The DevOps as a Service provider provides a toolset that covers all relevant aspects of the DevOps process and provides them as a unified platform.

A DevOps as a Service platform enables tracking and management of every action taken in the software delivery process.

It enables organizations to set up continuous integration / continuous delivery (CI/CD) systems and pipelines to increase development velocity and provide continuous feedback to developers.

24



DevOps as a Service

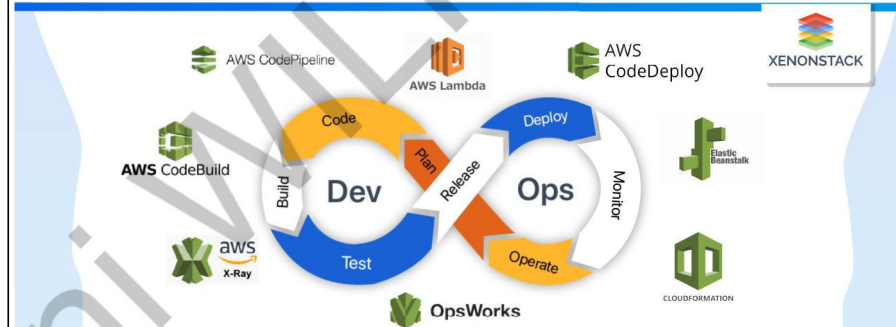
This platform approach hides the complexity of managing data and information flows across a complex DevOps toolchain.

Individuals and teams involved in the DevOps process can access any relevant technology without having to find, adopt, and learn multiple tools.

For example, a DevOps as a Service solution provides access to source code management (SCM), build servers, deployment management, and application performance management (APM) in one interface, with centralized auditing and reporting.

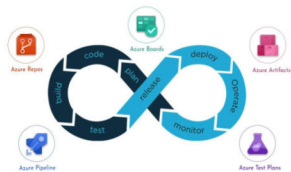
25

DevOps with AWS



26

DevOps with Azure



Azure Boards

Plan, track, and discuss work across teams, deliver value to your users faster.



Azure Repos

Unlimited cloud-hosted private Git repos. Collaborative pull requests, advanced file management, and more.



Azure Pipelines

CI/CD that works with any language, platform, and cloud. Connect to GitHub or any Git provider and deploy continuously to any cloud.



Azure Test Plans

The test management and exploratory testing toolkit that lets you ship with confidence.



Azure Artifacts

Create, host, and share packages. Easily add artifacts to CI/CD pipelines.

27

Thank you

12

28